

EXPERIMENT 2

NAME : Yug Nirwal
DATE : 18/08/2026
SAP ID: 59003354
SUBJECT : C programming

Purpose

To follow a small C program from source code to a working executable and observe the files created during each stage of compilation.

Environment

- Kali Linux
- Oracle VM VirtualBox
- C Programming language

AIM

To write a C program in kali Linux, save and compile it, observe the four compilation stages and their intermediate files, execute the final program, and verify the result.

C Source Program

The program itself is very simple so that the complete compilation can be followed easily.

```
#include <stdio.h>    //preprocessor directive to include builtin header file
#define PI 3.14       //PPD to define a symbolic constant or macro
int main(void)
{
    int radius;        //variable definition
    float area;
```

<code>printf("Enter the radius: ");</code>	<code>//printf() is builtin function to display output string</code>
<code>scanf("%d", &radius);</code>	<code>//scanf() is builtin function to get inputs</code>
<code>area = PI * radius * radius;</code>	<code>//compute</code>
<code>printf("Area = %f\n", area);</code>	
<code>return 0;</code>	
<code>}</code>	

How the program works

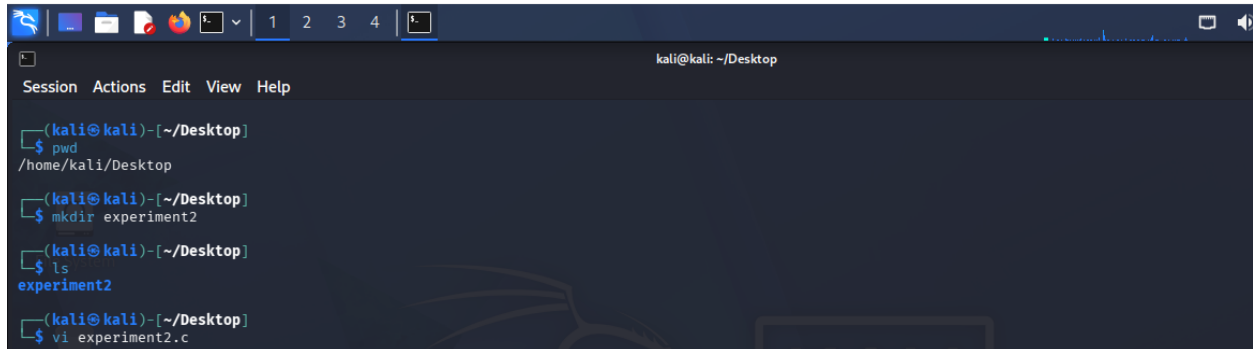
Statement	work in the program
<code>#include <stdio.h></code>	Makes the standard input/output functions <code>printf()</code> and <code>scanf()</code> available.
<code>#define PI 3.14</code>	Creates the symbolic constant <code>PI</code> used in the area formula.
<code>int radius</code>	Stores the whole-number radius entered by the user.
<code>float area</code>	Stores the calculated area, which may contain decimal values.
<code>scanf("%d", &radius)</code>	Reads the radius from the keyboard and stores it in <code>radius</code> .
<code>area = PI * radius * radius</code>	Applies the formula for the area of a circle.
<code>printf("Area= %f\n", area)</code>	Displays the final area. <code>%f</code> prints six digits after the decimal by default.

❏ Procedure: Creating and Running the Program

STEP 1 Start kali Linux and prepare the code

Virtual box was opened, the kali Linux was started, and a terminal was launched. The current directory was checked with “`pwd`”. A folder named “`experiment2`” was created, the directory were listed, and the source file was opened with `vi`.

```
pwd
mkdir experiment2
ls
vi experiment2.c
```



The screenshot shows a Kali Linux terminal window with the title bar 'kali@kali: ~/Desktop'. The terminal displays the following commands and their outputs:

```
(kali@kali)-[~/Desktop]
$ pwd
/home/kali/Desktop
(kali@kali)-[~/Desktop]
$ mkdir experiment2
(kali@kali)-[~/Desktop]
$ ls
experiment2
(kali@kali)-[~/Desktop]
$ vi experiment2.c
```

Figure 1: checking the path, creating the experiment folder and opening experiment2.c

STEP 2 Enter the code in kali

Kali opened the new file. Insert mode used so that the c program could be typed into the editor . The code includes the header file, the PI , variable declarations, input, calculation, output and return statement.

Pressing “i” in kali enters Insert mode. The bottom of the editor display -- INSERT --.

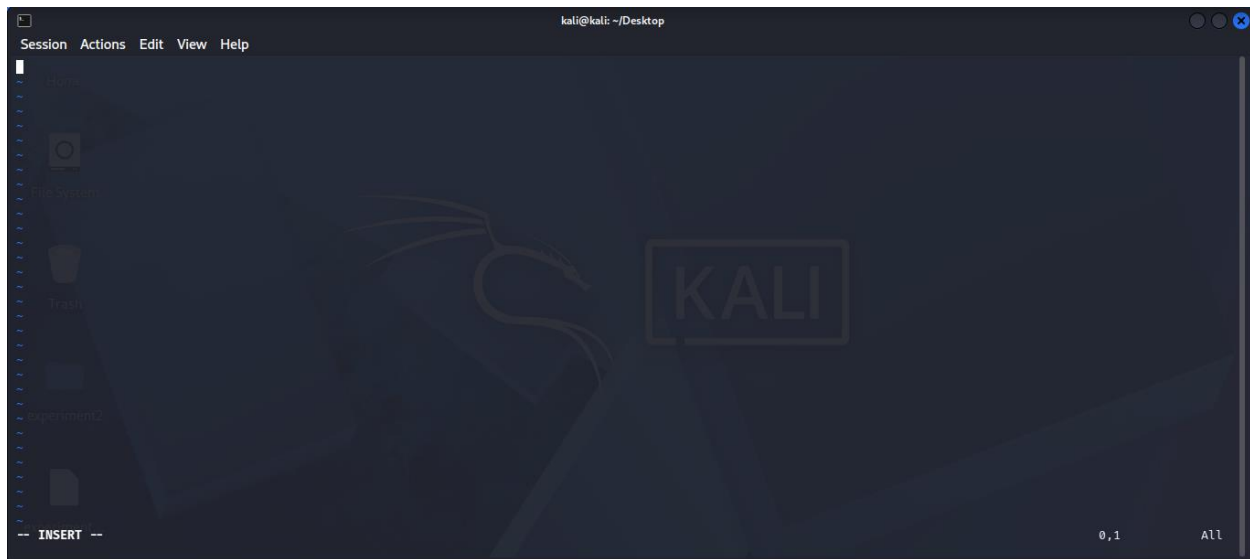


Figure 2: kali opened in Insert mode and ready for source-code

STEP 3 Enter the source code

The complete area of a circle program was entered. Comments were included beside important lines to describe the purpose of the preprocessor directives, variables, input, and computation.

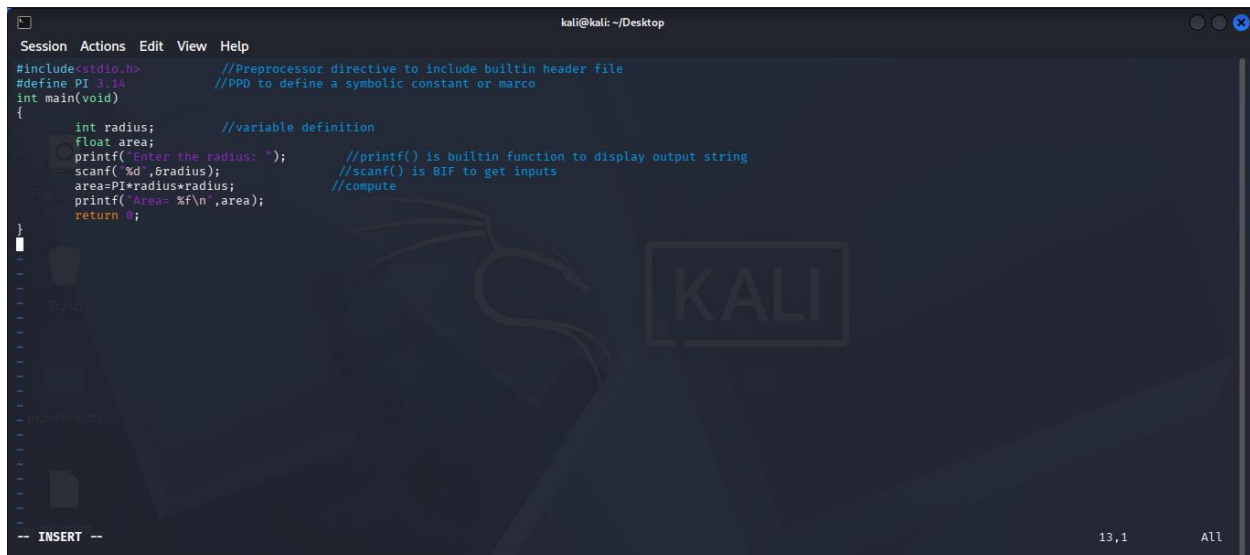


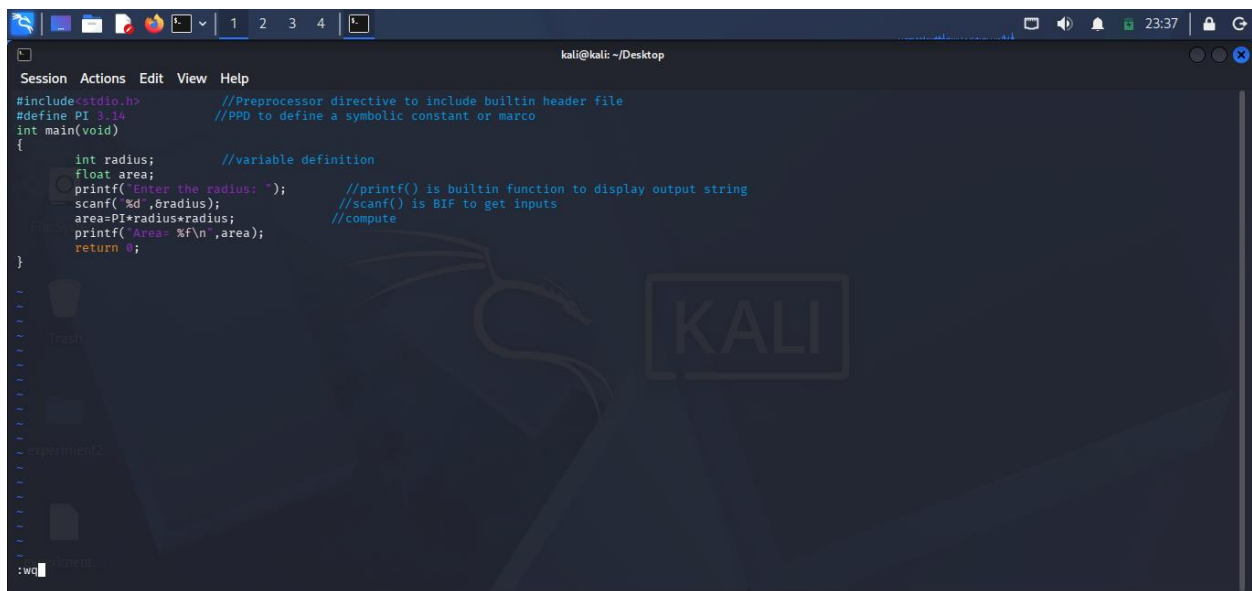
Figure 3: Complete C program entered in the editor.

STEP 4 Save the file and exit the editor

After typing the program, kali was switched from Insert mode to command mode. The file was written to disk and the editor was closed using the write-and-quit command.

```
Press Esc
:wq
Press Enter
```

In :wq, q means write (save) and q mean (quit). This is the required saving step before compilation.



```
kali@kali: ~/Desktop
Session Actions Edit View Help
#include<stdio.h>           //Preprocessor directive to include builtin header file
#define PI 3.14             //PPD to define a symbolic constant or marco
int main(void)
{
    int radius;             //variable definition
    float area;
    printf("Enter the radius: "); //printf() is builtin function to display output string
    scanf("%d",&radius);         //scanf() is BIF to get inputs
    area=PI*radius*radius;    //compute
    printf("Area= %f\n",area);
    return 0;
}

:wq
```

Figure 4: Saving experiment2.c and exiting with :wq.

STEP 5 Compile and execute the program

The C compiler driver was include was invoked cc. Because no output filename was supplied, the default executable was named a.out.

The executable was then run from the current directory using `./a.out`.
A radius of 7 was entered.

```
cc experiment2.c
./a.out
```

A compiler command that return without an error message normally indicated a successful build.

A screenshot of a terminal window with a dark background and a 'KALI' logo watermark. The terminal shows the following commands and output:

```
(kali@kali) ~/Desktop
$ cc experiment2.c
(kali@kali) ~/Desktop
$ ./a.out
Enter the radius: 7
Area= 153.860001
(kali@kali) ~/Desktop
$
```

Figure 5: Successful compilation and execution for radius(7).

STEP 6 Preserve the Intermediate compilation files

The `-save-temps` option was used so that the normally temporary preprocessed, assembly, and object files would remain available for inspection. The command was then used to verify the files.

```
cc -save-temps experiment2.c
ls
```

The saved files provide direct evidence of the preprocessing, compilation, assembly, and linking stages.

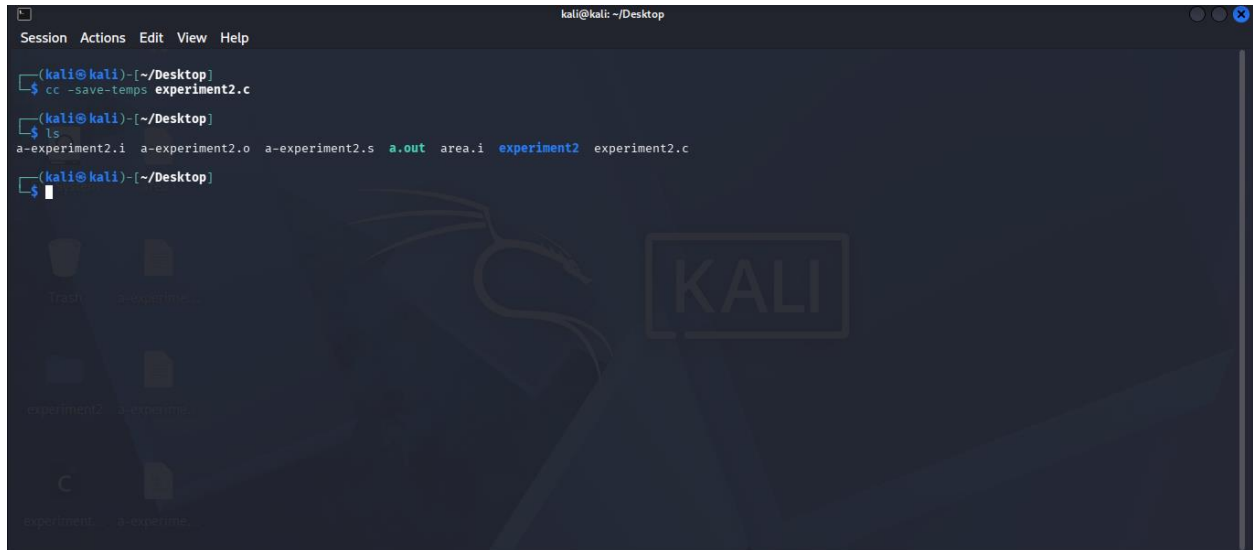
A screenshot of a Kali Linux terminal window. The window title is 'kali@kali: ~/Desktop'. The terminal shows the following commands and output:
1. Command: `cc -save-temps experiment2.c`
2. Command: `ls`
Output: `a-experiment2.i a-experiment2.o a-experiment2.s a.out area.i experiment2 experiment2.c`
The terminal background features a Kali Linux logo and the word 'KALI' in a large, stylized font.

Figure 6: Intermediate .i,.s,.o files saved alongside the final a.out executable.

04 The Four Stages of C Compilation

A single `cc` command performs several operations behind the scenes. The `-save-temps` option makes the intermediate results visible.

No.	Stage	Input	Main action	Output
1	Preprocessing	experiment2.c	Processes <code>#include</code> and <code>#define</code> directives, removes comments, and expands macros.	a-experiment2.i
2	Compilation	.i file	Checks and translates the preprocessed C program into assembly language.	a-experiment2.s
3	Assembly	.s file	Converts assembly instructions into	a-experiment2.o

			relocatable machine-code object data.	
4	Linking	.o file + libraries	Combines the object file with required library code such as printf() and scanf().	a.out

Stage 1 - Preprocessing

The preprocessor handles lines beginning with # before the C compiler analyzes the program. In this experiment, it brings in declarations from stdio.h and replaces every use of PI with 3.14. The saved output is the .i file.

```
cc -E experiment2.c -o experiment2.i
```

Equivalent stage-only command. In the screenshot, cc -save-temps produced a-experiment2.i automatically.

Stage 2 - Compilation

The compiler checks the preprocessed program and converts it into assembly instructions for the target system. This is where syntax and type-related errors are usually reported. The saved output is the .s file.

```
cc -S experiment2.i -o experiment2.s
```

Equivalent stage-only command for generating assembly language.

Stage 3 - Assembly

The assembler translates the assembly instructions into machine-code object data. The object file is not yet a complete runnable program because references to external library functions still need to be resolved.

```
cc -c experiment2.s -o experiment2.o
```

Equivalent stage-only command for creating the object file.

Stage 4 - Linking

The linker combines the object file with the required startup and standard-library code. It resolves references to functions such as printf() and scanf(), then produces the final executable. With no -o option, the default executable name is a.out.

```
cc experiment2.o -o a.out
```

Result

The experiment was completed successfully. The C program was created and saved in Vim, compiled with the cc command, and executed as ./a.out. For a radius of 7, the program displayed an area of 153.860001 square units, which agrees with the expected value of 153.86 using PI = 3.14. The cc -save-temps command

also preserved the .i, .s, and .o files, confirming the preprocessing, compilation, assembly, and linking stages.

Conclusion

This experiment showed the complete journey of a C program in a practical way. We began with a readable .c file, saved it correctly, and used the compiler to transform it through four stages into a runnable executable. Seeing the .i, .s, and .o files made the normally hidden compilation process easier to understand. The final output confirmed that both the program logic and the build process worked correctly in Kali Linux running on Oracle VM VirtualBox.